

Lecture: Geometry and Motion

- ▶ This lecture covers four topics.
- ▶ Basic Projection. Perspective. Vanishing Points.
- ▶ Camera Calibration. Geometry of Binocular Steropsis. Essential Matrix. Fundamental Matrix.
- ▶ Structure from Rigid Motion. Extension to Non-Rigid Motion.
- ▶ Geometric Priors. Manhattan World.

Geometry of Projection

- ▶ Most analysis of projection is based on the Pinhole camera model (perspective projection and approximations to it). Real cameras have lens of finite size (a pinhole camera has a lens of size zero). See Szeliski's book for corrections to the pinhole camera model (often required near boundaries of images).
- ▶ Major properties of perspective projection. Straight lines project to straight lines. Parallel lines in space project to lines that converge to a vanishing point (which may be at infinity).
- ▶ Basic equations for perspective projection. A camera is defined by an origin $\vec{o}$, right-handed coordinate axes $\vec{a}$, $\vec{b}$, $\vec{c}$, where $\vec{c}$ is the direction of gaze of the camera. $\vec{a}$, $\vec{b}$ are axes of the imaging plane, and a focal length f . The imaging plane is defined by the equation: $(\vec{r} - \vec{o}) \cdot \vec{c} = -f$.
- ▶ The projection of a point $\vec{r}$ onto the image plane is given by $u/f = -\frac{(\vec{r}-\vec{o}) \cdot \vec{a}}{(\vec{r}-\vec{o}) \cdot \vec{c}}$, $v/f = -\frac{(\vec{r}-\vec{o}) \cdot \vec{b}}{(\vec{r}-\vec{o}) \cdot \vec{c}}$. Here u, v are the projections in the directions $\vec{a}, \vec{b}$ in the imaging plane.

Geometry of Projection: Straight Lines and Derivation

- ▶ This can be derived as follows. For simplicity, we set $\vec{o} = 0$ (hence we replace $(\vec{r} - \vec{o})$ by $\vec{r}$). Draw a straight line $\vec{r}(\lambda) = \lambda \vec{r}$ from a point $\vec{r}$ in space which passes through the pinhole and interacts the imaging plane at the position $u\vec{a} + v\vec{b} - f\vec{c}$.
- ▶ To derive the formulas for u and v we solve $\vec{r}(\lambda^*) \cdot \vec{c} = -f$ (for where the line meets the imaging plane) to obtain $\lambda^* = -f/(\vec{r} \cdot \vec{c})$ and the projection point is at $\lambda^* \vec{r} = -f/(\vec{r} \cdot \vec{c}) \vec{r}$. To obtain u, v we take the dot products with respect to $\vec{a}, \vec{b}, \vec{c}$.
- ▶ A *straight line* is specified by $\vec{r}(\lambda) = \vec{r}_0 + \lambda \vec{t}_0$ where λ specifies the position on the line, $\vec{t}_0$ specifies the direction of the line, and $\vec{r}_0$ is a point that the line passes through (any point on the line can be used). A plane is specified by $\vec{r} \cdot \vec{n} = k$, where $\vec{n}$ is the surface normal to the plane ($|\vec{n}| = 1$), and k is the shortest distance of the plane to the origin ($= \min |\vec{r}|$ s.t., $\vec{r} \cdot \vec{n} = 0$).

Geometry of Projection: Parallel Lines and Vanishing Points

- ▶ Parallel lines in space can be expressed by $\vec{r}_0 + \lambda \vec{r}$ where $\vec{r}$ is a unit vector in the direction of the line, λ is the length, and different values of $\vec{r}_0$ specify different lines (all parallel to each other). (For simplicity we set $\vec{o} = \vec{0}$ in this section).
- ▶ Each line in space projects to a line in the image plane $u(\lambda) = \frac{-f(\vec{r}_0 + \lambda \vec{r}) \cdot \vec{a}}{(\vec{r}_0 + \lambda \vec{r}) \cdot \vec{c}}$,
 $v(\lambda) = \frac{-f(\vec{r}_0 + \lambda \vec{r}) \cdot \vec{b}}{(\vec{r}_0 + \lambda \vec{r}) \cdot \vec{c}}$.
- ▶ Without loss of generality, set $\vec{r}_0 \cdot \vec{c} = 0$. This can be done by setting $\vec{r}_0 \mapsto \vec{r}_0 - \frac{(\vec{r}_0 \cdot \vec{c})}{(\vec{r} \cdot \vec{c})} \vec{r}$. Note that $\vec{r} \cdot \vec{c} = 0$, only for lines perpendicular to the direction of gaze $\vec{c}$.
- ▶ Then $u(\lambda) = -f \frac{(\vec{r} \cdot \vec{a})}{(\vec{r} \cdot \vec{c})} - f \frac{(\vec{r}_0 \cdot \vec{a})}{(\vec{r} \cdot \vec{c})} \frac{1}{\lambda}$. $v(\lambda) = -f \frac{(\vec{r} \cdot \vec{b})}{(\vec{r} \cdot \vec{c})} - f \frac{(\vec{r}_0 \cdot \vec{b})}{(\vec{r} \cdot \vec{c})} \frac{1}{\lambda}$. Recall that λ is the distance along the line in 3D space so $1/\lambda$ is the inverse distance.

Vanishing Points

- ▶ Consider a family of parallel lines with direction $\vec{r}$ (and different values of $\vec{r}_0$). Then as we move along the lines to infinity. i.e. $\lambda \mapsto \infty$, then $1/\lambda \mapsto 0$. Hence $u(\lambda) \mapsto -f \frac{(\vec{r} \cdot \vec{a})}{(\vec{r} \cdot \vec{c})}$ and $v(\lambda) \mapsto -f \frac{(\vec{r} \cdot \vec{b})}{(\vec{r} \cdot \vec{c})}$, which is *independent of $\vec{r}_0$* .
- ▶ In other words, all the lines in direction $\vec{r}$ converge to the same *vanishing point*. The only exceptions are lines for which $\vec{r} \cdot \vec{c} = 0$, i.e. those lines which are perpendicular to the direction of gaze $\vec{c}$.
- ▶ The number of vanishing points depends on how many families of parallel lines there are in the image. Some images will have no vanishing points. Images in Manhattan can have three vanishing points.
- ▶ We can create an image with as many vanishing points as we want – e.g., by placing a large number of square tables in a room so that each table is not aligned to any of the others, The number of vanishing points will be twice the number of tables plus one – if we assume that the floor is flat.

The Ground Plane

- ▶ In the real world there is often a flat ground plane and the viewpoint $\vec{c}$ is perpendicular to it (i.e. the direction of gaze is parallel to the ground plane).
- ▶ In this case. lines which are perpendicular to the ground plane (e.g., trees growing vertically from the ground) will be orthogonal to the viewpoint $\vec{r} \cdot \vec{c} = 0$, and hence the vanishing points of these lines will be at infinity (i.e. trees will tend to be parallel in the image as well as is the viewed scene).
- ▶ Ultimately this is due to physics. Humans and animals typically stand vertically to the ground plane in order to resist gravity.
- ▶ Animals, and Computer Vision systems, exploit the ground plane to estimate the depth of objects. This can be used by filmmakers to make some people appear to much bigger than other people (e.g., the main character in the Green Mile, Gandalf in Lord of the Rings.)

Linear Projection Approximations

- ▶ Perspective projection can often be approximated by scaled orthographic project, provided the distance $\vec{r} \cdot \vec{c}$ is large and does not vary very much for points $\vec{r}$ in the viewed scene.
- ▶ This can be seen by doing Taylor series approximations of the perspective projection equations and, even simpler, to approximate $\vec{r} \cdot \vec{c}$ by a constant. In this case, we obtain scaled orthographic projection $u = -f(\vec{r} \cdot \vec{a})$, $v = -f(\vec{r} \cdot \vec{b})$. Here f is the scale.
- ▶ The advantage of linear projection approximations is that they simplify the mathematics, see later this lecture, but they also throw away information.
- ▶ A consequence of linear projection is that we no longer have vanishing points. For any image, you can visually check to see whether parallel lines in the viewed scene appear parallel in the image. If so, linear projection approximations are probably valid.

Linear Projection Approximations: Affine Cameras

- ▶ The most general form of linear projection is

$$(u, v) = \begin{pmatrix} K_1 & K_2 & K_3 \\ H_1 & H_2 & H_3 \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} \quad (12)$$

- ▶ To get scaled orthographic we set $\vec{K} \cdot \vec{H} = 0$ with $|\vec{K}| = |\vec{H}|$. Where $\vec{K} = (K_1, K_2, K_3)$ and $\vec{H} = (H_1, H_2, H_3)$.
- ▶ People write papers, and entire book chapters, about special cases with restrictions on $\vec{K}$ and $\vec{H}$.

Camera Calibration

- ▶ The goal of camera calibration is to find the relative positions and viewpoints of different cameras. These are prerequisites for estimating depth from binocular stereo, structure from motion, or depth from multiple cameras. Calibration is also needed for many other tasks (e.g., Gaussian splatting models of lighting). The human visual system also has to calibrate itself, perhaps by using these methods.
- ▶ Camera calibration is a classic computer vision problem because it could be studied mathematically. It assumes that it is possible to estimate points that correspond between the two images. There is a big literature on how to do this using interest points and corner detectors (beyond the scope of this course).
- ▶ Mathematical analysis resulted in several theorems informally called M views of N points. The idea was to find the minimum number of points that could be used.
- ▶ Here we restrict to describing the essential matrix and the fundamental matrix and use perspective projection. We also introduce and use homogeneous coordinates.

The Essential Matrix

- Suppose we have a point which is given by $\vec{x} = (X_1, X_2, X_3)$ in one coordinate system (from one camera) and by $\vec{x}' = (X'_1, X'_2, X'_3)$ in a second coordinate system (second camera). Its projection onto the two cameras is given by:

$$\begin{pmatrix} u \\ v \end{pmatrix} = \frac{f}{X_3} \begin{pmatrix} X_1 \\ X_2 \end{pmatrix} \quad \begin{pmatrix} u' \\ v' \end{pmatrix} = \frac{f}{X'_3} \begin{pmatrix} X'_1 \\ X'_2 \end{pmatrix} \quad (13)$$

- In homogeneous coordinates this is expressed as

$$\vec{u} \doteq \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \frac{f}{X_3} \begin{pmatrix} X_1 \\ X_2 \\ X_3 \end{pmatrix} \quad \vec{u}' \doteq \begin{pmatrix} u' \\ v' \\ 1 \end{pmatrix} = \frac{f}{X'_3} \begin{pmatrix} X'_1 \\ X'_2 \\ X'_3 \end{pmatrix} \quad (14)$$

- The coordinates $\vec{x}$ and $\vec{x}'$ are related by a rotation matrix $\vec{R}$ and a translation vector $\vec{t}$, which relates the two cameras, and is given by $\vec{x} = \vec{R}\vec{x}' + \vec{t}$.

Essential Matrix

- ▶ Define $\vec{E} = \vec{R}[\vec{t}]_x$ (matrix product between the rotation matrix $\vec{R}$ and the matrix $[\vec{t}]_x$). Here $[\vec{t}]_x$ is the matrix with components $\sum_k \epsilon_{ijk} t_k$, where ϵ_{ijk} is the anti-symmetric tensor. I.e. $\epsilon_{123} = 1$ and $\epsilon_{ijk} = -\epsilon_{jik}$ and $\epsilon_{ijk} = -\epsilon_{ikj}$. This implies $\epsilon_{ijk} = 0$ if any of i, j, k are identical, and $\epsilon_{ijk} = 1$ if i, j, k are even permutations of 1, 2, 3 and $\epsilon_{ijk} = -1$ for odd permutations of 1, 2, 3.
- ▶ We claim that $\vec{u}'^T \vec{E} \vec{u} = 0$, where T denotes matrix transpose. This imposes a constraint on $\vec{r}$ and $\vec{t}$. This implies that for each corresponding point between the two cameras, we get one equation for the camera parameters $\vec{r}$ and $\vec{t}$.
- ▶ Proof. $\vec{u}'^T \vec{E} \vec{u} \propto \vec{X}'^T \vec{E} \vec{X} = (\vec{X} - \vec{t})^T \vec{R}^T \vec{R} [\vec{t}]_x \vec{X} = (\vec{x} - \vec{t})^T [\vec{t}]_x \vec{X} = 0$. This follows because $\vec{X}' = \vec{R}(\vec{x} - \vec{t})$ and because $\sum_{ijk} \epsilon_{ijk} t_k x_j = \sum_{ijk} \epsilon_{ijk} x_i x_j t_k - \sum_{ijk} t_i x_j t_k = 0$.
- ▶ There are six parameters $\vec{R}, \vec{t}$ needed to calibrate the camera. So if we have six corresponding points that lie in general position (i.e. are not aligned in any unlikely way) then we have enough equations to solve for them.

Structure from Rigid Motion

- ▶ Structure from rigid motion assumes that we have a set of images/views of a rigid scene or rigid object. In its simplest form, we assume that we can detect a set of interest points in each image and can correspond them between the different images. We will calibrate the cameras – estimate the different viewpoints – and estimate the shape of the scene or object.
- ▶ We assume linear projection. We represent the object by a set of N points $\{(X_1^n, X_2^n, X_3^n) : n = 1, \dots, N\}$ (interest points which can be detected and matched). We have M views which correspond to projection matrices:

$$\begin{pmatrix} \vec{K}^n \\ \vec{H}^n \end{pmatrix} = \begin{pmatrix} K_1^m & K_2^m & K_3^m \\ H_1^m & H_2^m & H_3^m \end{pmatrix} \text{ for } m = 1, \dots, M.$$

- ▶ We have observations in the images: $u^{n,m} = \vec{K}^m \cdot \vec{X}^n = \sum_{i=1}^3 K_i^m X_i^n$ and $v^{n,m} = \vec{H}^m \cdot \vec{X}^n = \sum_{i=1}^3 H_i^m X_i^n$.
- ▶ These imply that the matrices $u^{n,m}$ and $v^{n,m}$ are of rank three. I.e. the set of image points of an object lie in a three-dimensional space (Basri and Ullman).

Structure from Rigid Motion (2)

- ▶ Estimating the structure requires finding the shape $\{\vec{X}^n\}$ and the viewpoints $\{(\vec{K}^m, \vec{H}^n)\}$. We assume the observations are corrupted by additive zero mean Gaussian noise. We formulate by minimizing:

$$E[K, H, X] = \sum_{n,m} (u^{n,m} - \sum_{i=1}^3 K_i^m X_i^n)^2 + \sum_{n,m} (v^{n,m} - \sum_{i=1}^3 H_i^m X_i^n)^2$$
 with respect to K, H, X . Note there is an ambiguity because we can rotate the object and rotate the coordinate system without affecting the solution.
- ▶ First consider the simpler case. $E(K, X) = \sum_{n,m} (u^{n,m} - \sum_{i=1}^3 K_i^m X_i^n)^2$. This is a bilinear problem. If K is known, then the solution for X is linear. If X is known, then the solution for K is linear.
- ▶ The global minimum can be solved for by Singular Value Decomposition (SVD). (Alternatively, can do steepest descent on $E[K, X]$. This is non-convex but has no local minima but only saddle points).
- ▶ We use SVD to express $\vec{U} = \vec{E} \vec{D} \vec{F}^T$ where $\vec{D}$ is a diagonal matrix ($D_{ij} = d_i \delta_{ij}$ where $\delta_{ij} = 1$ if $i = j$ and $= 0$ otherwise). The matrices $\vec{E}$ and $\vec{F}$ are orthogonal, so $\vec{E} \vec{E}^T = \vec{I} = \vec{F} \vec{F}^T$, where $\vec{I}$ is the identity matrix. The columns of $\vec{E}$ and $\vec{F}$ correspond to the eigenvectors of $\vec{U} \vec{U}^T$ and $\vec{U}^T \vec{U}$ respectively.

Structure from Rigid Motion (3)

- ▶ Let $e_k(m)$ and $f_k(n)$ be the first three columns of $\vec{U}$ and $\vec{V}$.
- ▶ Then the solutions are of form $K_i^m = \sum_{k=1}^3 P_{ik} e_k(m)$ and $X_i^n = \sum_{k=1}^3 Q_{ik} f_k(n)$. Where $\vec{P}\vec{Q}^T = \vec{D}_3$ (where D_3 is the first three-by-three block of $\vec{D}$ (organized so that $d_1 \geq d_2 \geq d_3 \dots$).
- ▶ This has an ambiguity $\vec{P} \mapsto \vec{P}\vec{A}$ and $\vec{Q}^T \mapsto \vec{A}^{-1}\vec{Q}^T$. Here $\vec{A}$ is any invertible matrix. Part of this ambiguity the coordinate transformation (i.e. rotation) described earlier.
- ▶ We get a similar solution for $\vec{H}$ and $\vec{X}$ by minimizing the second term $\sum_{n,m} (v^{n,m} - \sum_{i=1}^3 H_i^m X_i^n)^2$. If we impose further constraints on the projection matrix, i.e. that $\vec{K}^m \cdot \vec{H}^m = 0$ for all m , then we obtain a unique solutions for $\vec{X}, \vec{K}, \vec{H}$ up to the rotation ambiguity (coordinate transform).
- ▶ This algorithm was reinvented. Kontsevich et al. Tomasi and Kanade. It can be extended to perspective projection, but require a more complex algorithm.

Structure from Non-Rigid Motion

- ▶ This can be extended to a spacial class of non-rigid motion if we require that the object is expressed in terms of a linear sum of basis vectors, where the coefficients of the basis vectors is a function of time.
- ▶ More specifically, $\vec{X}^m = \sum_{i=1}^d \alpha_i^m \vec{b}_i$ where $\{\vec{b}_i : i = 1, \dots, d\}$ are the basis functions and the α_i^m are the coefficients (which are functions of time frame m).
- ▶ This approach was developed by Bregler et al. (CVPR 2000). Initially it was poorly understood and researchers made unnecessary assumptions because they thought there were ambiguities in the $\vec{b}, \alpha, \vec{K}, \vec{H}$. But it was realized that these were only coordinate changes. This led to theoretical very attractive algorithms (Y. Dai et al. CVPR 2012). But they are complex, are not guaranteed to converge to the global minimum, and only apply to limited types of nonlinear motion. For example, they cannot deal with articulated objects like humans and animals.

World Geometry

- ▶ European painter exploited vanishing points to give their pictures the impression of three-dimensional depth (starting from crude attempt by Giotto). Computer vision researchers exploited this to determine three-dimensional structure from objects with sufficient vanishing points.
- ▶ Many viewed scenes contain a fixed ground plane. Hence objects lying on this plane, e.g., trees, cars, people, typically are oriented so one of their major axis is perpendicular to the plane (this is consequence of gravity). Hence detecting the ground plane is important and vanishing points can be used to help detect it.
- ▶ Humans use these cues to obtain depth from perspective. This term is slightly misleading because these cues require making strong assumptions about the viewed scene. If these assumptions are invalid then human perceptions makes mistakes. For example, the Ames room looks like a real room but has different geometry. This fools humans, so we see people change in size when they moves in this room.

Manhattan World

- ▶ Manmade scenes often have three dominant axes which are perpendicular to each other, this has been called *Manhattan World*. It can be exploited to estimate the orientation of the viewer/camera with respect to this natural coordinate system of the scene. This can be done either by finding vanishing points, or more directly from the statistics of the edge orientations in the image (assuming that most of them come from the three dominant axes).
- ▶ The idea of Manhattan world has been extended to say that the world often consists of planar surfaces which are aligned to these three major axes (and this has been used as a prior for binocular stereo).
- ▶ This relates to the classic *blocks world* which is constructed by placing a set of blocks on the ground. It was shown that by analyzing the boundaries of these blocks it was possible to determine the three-dimensional structure. This was extended to more realistic situations in cities. More generally if the structure of the world is generated by a grammar it is often possible to use this to inversely determine the three-dimensional structure of the scene (provided the grammar is sufficiently regular). To summarize, if the view scene is structured enough then it is often possible to estimate much of its geometry from a single image.